Parsing Sentences using Recursive Neural Networks

Rukmangadh Sai Myana - 160070051
Ritesh Goru - 160070048
Devesh Kumar - 16D070044
Pranav Deo - 170040012

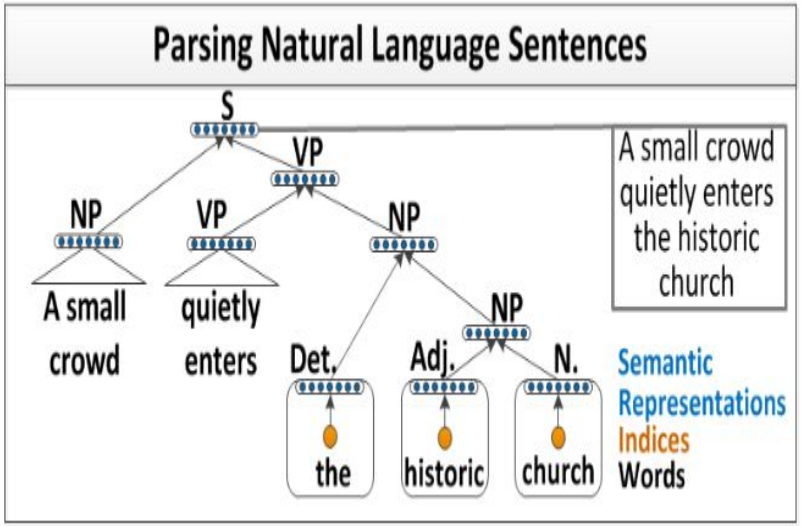
Dataset - Penn Treebank

- 4 forms of sentences in the Penn Treebank:
  - **Raw form:** This is just the original form of the sentence
  - **Tagged form:** In this form, the sentence is divided into its corresponding words and each word is given a Part-of-Speech(POS) tag.
  - **Parsed form:** In this form, the sentence is parsed into its phrasal categories
  - **Combined form:** In this form, the sentence is tagged both with phrasal categories and POS tags
- We use the Tagged and Parsed forms of the sentences for our algorithm here
- The tagged forms are used for predicting the part of speech(POS) of each leaf node while the parsed form is used for predicting the label of each phrase

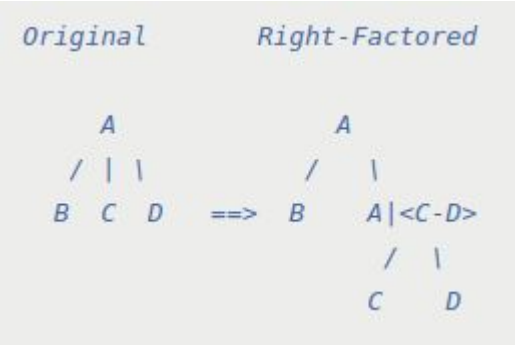
Data Pre-Processing

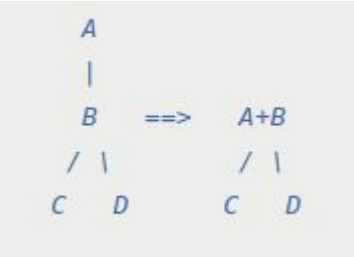
Binarization of parse trees - I

- We reduce the number of categories into **six** - eg **NP (Noun Phrase), VP (Verb Phrase), ADJP (Adjective Phrase), PP (Preposition Phrase), S (Simple Declarative Clause), X (Others)**
- To do this, we convert the Penn Treebank into a binary tree as we are using a greedy approach and it is easy to compare two binary subtrees etc



Binarization of parse trees - II





Chomsky Normal Form (CNF)

The CNF algorithm binarizes the tree. In order to convert a tree into CNF, we simply need to ensure that every subtree has either two subtrees as children (binarization), or one leaf node (non-terminal). In order to binarize a subtree with more than two children, we must introduce artificial nodes

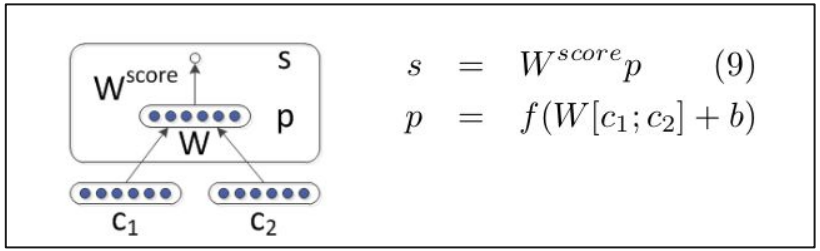
Unary Collapsing

Collapse unary productions (ie. subtrees with a single child) into a new non-terminal (Tree node). This is useful when working with algorithms that do not allow unary productions, yet you do not wish to lose the parent information

Algorithm

Algorithm

- The parsing algorithm has the following parts:
  - **Word Embedding:** Pre-trained word embeddings are used for embedding the vocabulary of the Penn treebank into a R^{300} vector space
  - **Neural Network for Parsing:** We are gonna use a neural network recursively for parsing the sentences. The structure of the neural network is given.



$$s = W^{score} p \quad (9)$$
$$p = f(W[c_1; c_2] + b)$$

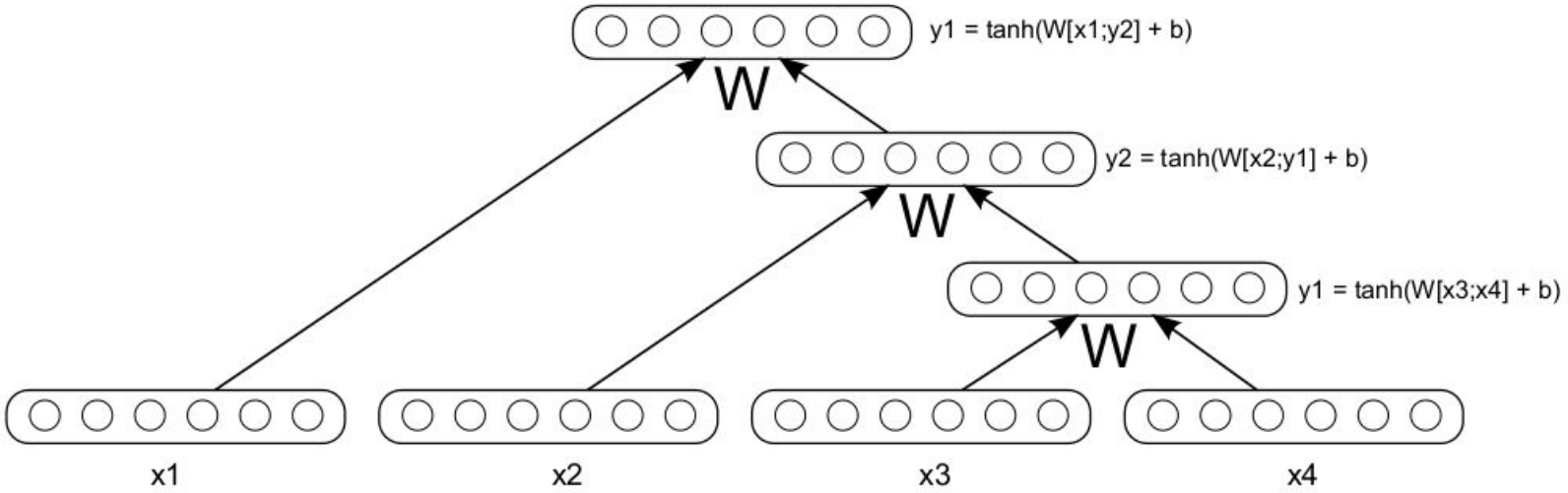


Figure 1: An example tree with a simple Recursive Neural Network: The same weight matrix is replicated and used to compute all non-leaf node representations. Leaf nodes are n-dimensional vector representations of words.

Objective Function - Tree building Neural Net

$$J(\theta) = \sum_{i=1}^{i=M} (s(x_i, y_i) - (s(x_i, y_{greedy}) + \triangle(x_i, y_{greedy}))) \quad (1)$$

Where,

y_i = ground truth tree

M = Size of the training dataset

$s(x_i, y_i)$ = score of the ground truth tree with the parameter values - θ for the neural network

$s(x_i, y_{greedy})$ = score of the greedy tree built using the neural network with the parameter values - θ for the neural network

$\triangle(x_i, y_{greedy})$ = Penalization of the neural network for mis-parsing the sentence

Label Prediction Neural Network

- We have 6 possible labels for each phrase - **NP, VP, PP, ADJP, S, X**
- **NP - Noun Phrase, VP - Verb Phrase, PP - Preposition Phrase, ADJP - Adjective Phrase, S = Simple Declarative Clause, X = others**
- The label prediction is done by the adding to each RNN parent node (after removing the scoring layer) a **simple softmax layer** to predict class labels, such as visual or syntactic categories label.

$$label_p = softmax(W^{label} p).$$

- The objective function in this case is the cross-entropy loss evaluated wrt to the labels for the gt tree

Algorithm Learning and Tree Prediction

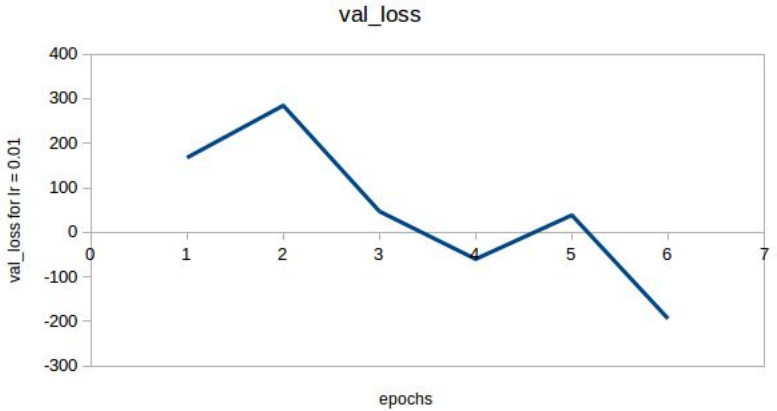
- We calculated the -1*J function which is described above and added the ground truth cross entropy loss
- The combined loss function is **minimised** using sgd optimiser in pytorch

Combined Loss Function = -J + (cross-entropy loss)

- The Tree of any new sentence is obtained by passing it through the Tree building Neural Network

Results


train_loss
train_loss for lr =0.01
900
800
700
600
500
400
300
200
100
0
0
1
2
3
4
5
6
7
epochs


val_loss
val_loss for lr = 0.01
400
300
200
100
0
-100
-200
-300
0
1
2
3
4
5
6
7
epochs